NOHACK

TECHNICAL ARCHITECTURE

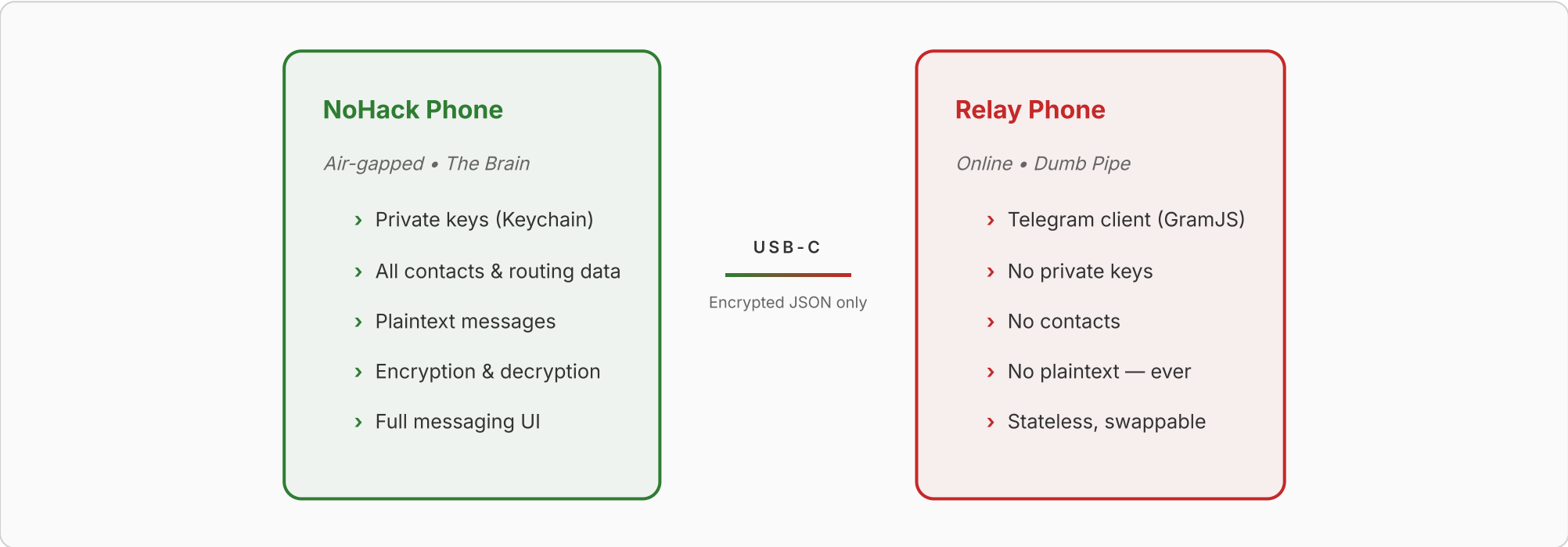
Plaintext never exists on a hackable device.
Air-gapped messaging that eliminates the remote attack surface entirely.

v3.0 — Confidential — March 2026

System Overview

NoHack is a paired-device messaging system. The **NoHack phone** is air-gapped — permanently offline, holding all keys, contacts, and plaintext. The **Relay phone** is online, connecting to Telegram as a stateless, encrypted pipe. The two phones connect via a USB-C cable. No setup required — plug in and it works.

Device Architecture



Zero Trust Relay

The security model assumes the Relay is **fully compromised**. It has internet access, runs Telegram, and is exposed to all remote attack vectors. Even with full Relay access, an attacker cannot read, forge, or tamper with messages.

Distribution

Getting Started with NoHack

The NoHack app is pre-installed on the air-gapped device. To set up the Relay:

1. **Scan QR Code**
   NoHack displays a QR code linking to **nohack.app**
2. **Install Relay**
   Download the Relay APK directly — no Play Store needed
3. **Connect USB**
   Plug in a USB-C cable. Auto-connects, zero configuration
4. **Message**
   Start messaging. All encryption happens on the NoHack phone

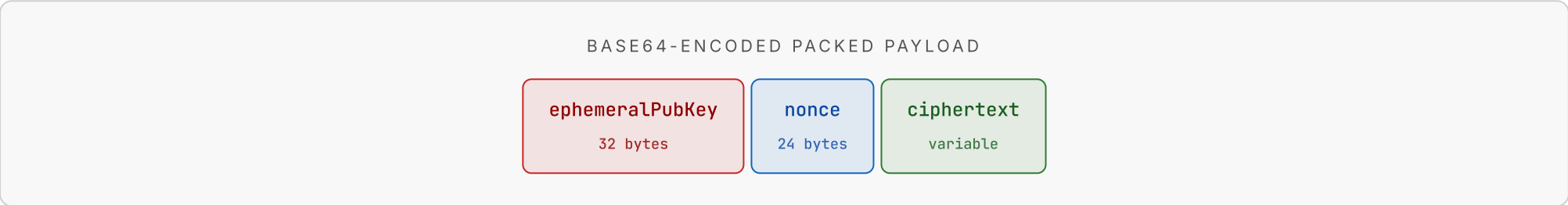
Encryption

All messages are encrypted using **NaCl box** (Curve25519 + XSalsa20-Poly1305) via the `tweetnacl` library. Each message uses a fresh ephemeral keypair for forward secrecy.

Key Generation

On first launch, NoHack generates a Curve25519 keypair. The public key is stored in AsyncStorage. The secret key is stored in the Android Keychain (with an AsyncStorage fallback).

Encrypted Payload Format



Forward Secrecy

Each outgoing message generates a **fresh ephemeral Curve25519 keypair**. The ephemeral secret key is used once for encryption and then discarded. Even if the sender's long-term key is compromised, past messages remain secure.

Decryption

NoHack extracts the ephemeral public key and nonce from the packed payload, then performs `nacl.box.open` with its own secret key. If decryption fails (wrong recipient), it throws a `DecryptionError` and silently drops the message.

What the Relay Sees

Only the base64-encoded encrypted blob. The Relay **cannot** decrypt messages (no secret key), forge messages (no sender's secret key), or read any content at any stage.

Message Protocol

.nohack v3 Format

Every message is a JSON object with protocol version, unique ID, and tracking tag:

FIELD	DESCRIPTION
`nohack`	Protocol version, always `"3"`
`id`	Unique message ID (timestamp base36 + random)
`tag`	3-letter tracking code (A-Z, no I/O) for log correlation
`type`	`"introduction"`, `"ack"`, or absent (encrypted message)
`payload`	Base64-encoded encrypted content
`senderPublicKey`	Sender's NaCl public key (base64)
`recipient`	Recipient's public key
`recipientTelegramId`	Routing: which Telegram user to deliver to
`relayTelegramId`	Attached by Relay: its own Telegram username

Inner Payload (after decryption)

```
{
  "type": "message",
  "senderPublicKey": "base64...",
  "contentType": "text | image | voice | video | mixed",
  "body": "..."
}
```

USB Communication

The two phones communicate via **Android Open Accessory (AOA)** protocol over USB-C. The Relay acts as USB Host and initiates the AOA handshake. NoHack acts as USB Accessory and responds.

Transport Protocol

Newline-delimited JSON over USB bulk transfer (16KB chunks).

COMMAND	DIRECTION	PURPOSE
`decrypt`	Relay → NoHack	Deliver encrypted message
`introduction`	Relay → NoHack	Deliver contact card
`ping`	Relay → NoHack	Check if NoHack is alive
`ack`	NoHack → Relay	Acknowledge ping / read receipts
`encrypted`	NoHack → Relay	Send encrypted message out

Data Flow

```
Relay (Host)                          NoHack (Accessory)
UsbHostModule.kt ──────────────────► UsbConnectionModule.kt
   bulkTransfer()                        inputStream.read()
   (16KB chunks)                         (16KB buffer)
        |                                      |
```

```
        ▼                                   ▼
RelayUsbService.ts                  UsbService.ts
   write()                           handleData()
   newline-delimited                  extractLines()
   JSON                               parseTransportLine()
        |                                   |
        ▼                                   ▼
RelayMainScreen.tsx                 ChatListScreen.tsx
   message router                    message router
```

USB Hardening

USB Device Filter

Only accepts `manufacturer="NoHack"` `model="NoHack Relay"`. No wildcard fallback — unknown USB devices are ignored entirely.

Buffer Cap

1 MB maximum buffer size. If exceeded, the entire buffer is dropped and the connection is reset. Prevents memory exhaustion attacks.

Command Whitelist

Only 3 commands accepted: `decrypt`, `introduction`, `ping`. All other commands are silently dropped.

No Code Execution

No `eval()`, no dynamic `require()`, no shell access. USB data is parsed as JSON only — never executed.

Routing Architecture

NoHack stores all contact and routing data. The Relay is completely stateless — zero contacts, zero routing tables. Relays can be swapped anytime without losing any data.

Contacts

FIELD	PURPOSE	STORED ON
`publicKey`	NaCl public key (base64), used as contact ID	NoHack only
`name`	Deterministic name from `keyToName()`	NoHack only
`telegramId`	Contact's relay Telegram username (for routing)	NoHack only

Outgoing Message Flow

1. NoHack encrypts message with recipient's public key
2. NoHack attaches `recipientTelegramId` from contact store
3. NoHack sends encrypted envelope via USB to Relay
4. Relay reads `recipientTelegramId`, strips it, attaches own `relayTelegramId`
5. Relay sends `.nohack` file to recipient's Telegram

Incoming Message Flow

1. Telegram message arrives at Relay (via NewMessage event)
2. Relay validates `.nohack v3` format, forwards to NoHack via USB
3. NoHack attempts decryption with its secret key
4. Success: stores message, extracts sender info, saves `relayTelegramId`
5. Failure: silently drops (message not addressed to this device)

Acknowledgement System

○	✓	✓✓	✓✓
Queued	**Sent**	**Received**	**Read**
Not yet sent (USB disconnected)	Delivered to Relay	Delivered to recipient's NoHack	Recipient opened the chat

Read Receipt Hardening

Read ACKs are only sent when `AppState === 'active'` (screen on, app in foreground). Messages received while the screen is off get grey ✓✓ (received) but **not** blue ✓✓ (read) until the user actually views them.

Security Model

The Relay phone is assumed **fully compromisable**. It has internet access, runs Telegram, and faces all remote attack vectors. The security model is designed so that even total Relay compromise cannot breach message confidentiality.

What a Compromised Relay CANNOT Do

Read Messages

NaCl box encryption. Relay has no secret key — only sees opaque base64 blobs.

Forge Messages

Inner payload includes `senderPublicKey`. Forging requires the sender's secret key.

Execute Code on NoHack

USB protocol only accepts 3 JSON commands. No eval, no shell, no dynamic code loading.

Access NoHack Filesystem

Android Accessory mode provides app-level I/O only. No filesystem, no ADB, no MTP/PTP.

Install Malware on NoHack

Accessory USB mode cannot push APKs, access ADB, or modify the system.

Brute-force Decryption

Ephemeral keys per message. Curve25519 provides ~128-bit security level.

What a Compromised Relay CAN Do (Mitigated)

Inject Fake Introduction

New contacts show verification warning: *"Verify this person out-of-band before sharing secrets."*

Redirect Replies

`relayTelegramId` changes are detected and logged with `console.warn`.

Impersonate Relay via USB

USB filter requires exact `manufacturer` and `model` match.

Memory Exhaustion

USB buffer hard-capped at 1 MB. Excess data is dropped entirely.

Trust Assumptions

#	ASSUMPTION	RATIONALE
1	**Physical USB cable**	Assumed passive — not a malicious hardware implant
2	**First contact exchange**	Initial key exchange requires out-of-band verification
3	**Android OS on NoHack**	Trusted (offline = no remote compromise vector; physical access is a separate risk)
4	**Relay swappability**	Relay can be replaced anytime; NoHack retains all contacts and keys

Telegram Integration

The Relay connects to Telegram using **GramJS** (`telegram` npm package) via WebSocket transport on Android.

ASPECT	DETAIL
Library	GramJS v2.26.22 with Hermes compatibility patch
Transport	WebSocket (`useWSS: true` , `ConnectionTCPObfuscated`)
Outgoing	Sent as `.nohack` file attachment via `client.sendFile()`
Incoming	Received via `NewMessage` event handler
Polyfill	`Symbol.asyncIterator` polyfilled for Hermes engine

Build System

Two Android product flavors built from the same codebase:

FLAVOR	PACKAGE	ENTRY POINT
nohack	`com.nohack`	`index.js` → `App.tsx`
relay	`com.nohack.relay`	`index.relay.js` → `RelayApp.tsx`

Data Persistence

DATA	STORAGE	DEVICE
Private key	Android Keychain + AsyncStorage fallback	NoHack only
Public key	AsyncStorage	NoHack only
Contacts	AsyncStorage (`@nohack_contacts`)	NoHack only
Messages	AsyncStorage (per-contact arrays)	NoHack only
Telegram session	AsyncStorage	Relay only
Everything else	—	Relay stores **nothing**

Contact Identification

`keyToName(publicKey)` generates a deterministic human-readable name from a public key hash:

EXAMPLE

```
hash(publicKey) → "BrightFox482"
```

Format: Adjective + Noun + 3 digits • Same key always produces the same name

Not Authentication

`keyToName()` is a **convenience label**, not authentication. Users must verify contacts through a separate, out-of-band channel before sharing sensitive information.